pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

Tensors and Gradients

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

One idea, repeated

Week 1: One neuron to a network

Gradients, layers, batches, a case study

Week 2: Training

Optimizers, dropout, class weights

Week 3: Sequences

Walk-forward on the Treasury curve

The loop inside every fit

1. Compute the loss on the rows
2. Work out the gradient of each weight
3. **Move each weight a little downhill**
4. Repeat

Today the loop is done by hand, for one neuron, and checked against TensorFlow.

Where Courses 4 and 5 left the card file

0.7190

Course 4 logistic regression

0.7725

Course 5 LightGBM

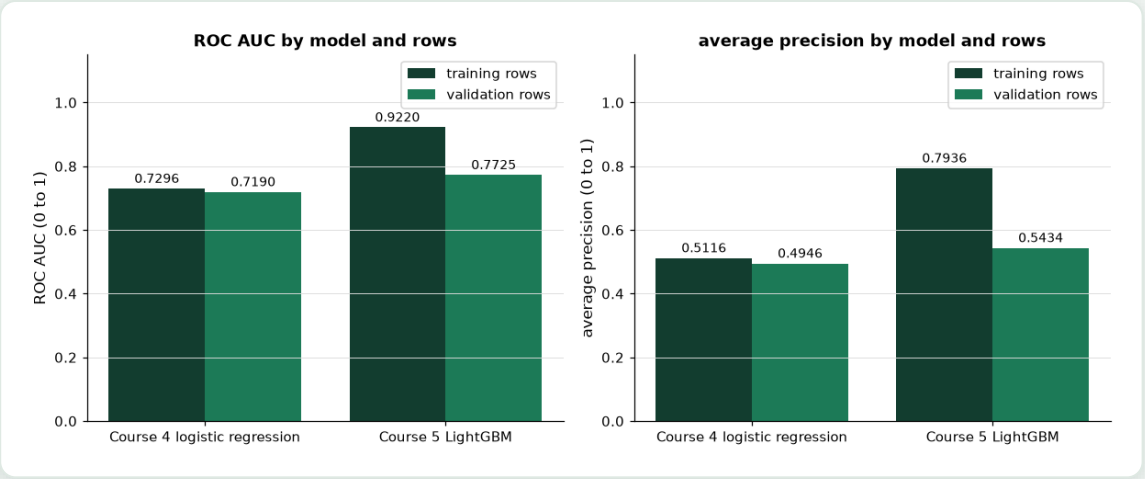
Validation ROC AUC; average precision 0.4946 and 0.5434. 18,000 training rows fit, 6,000 validation rows report.

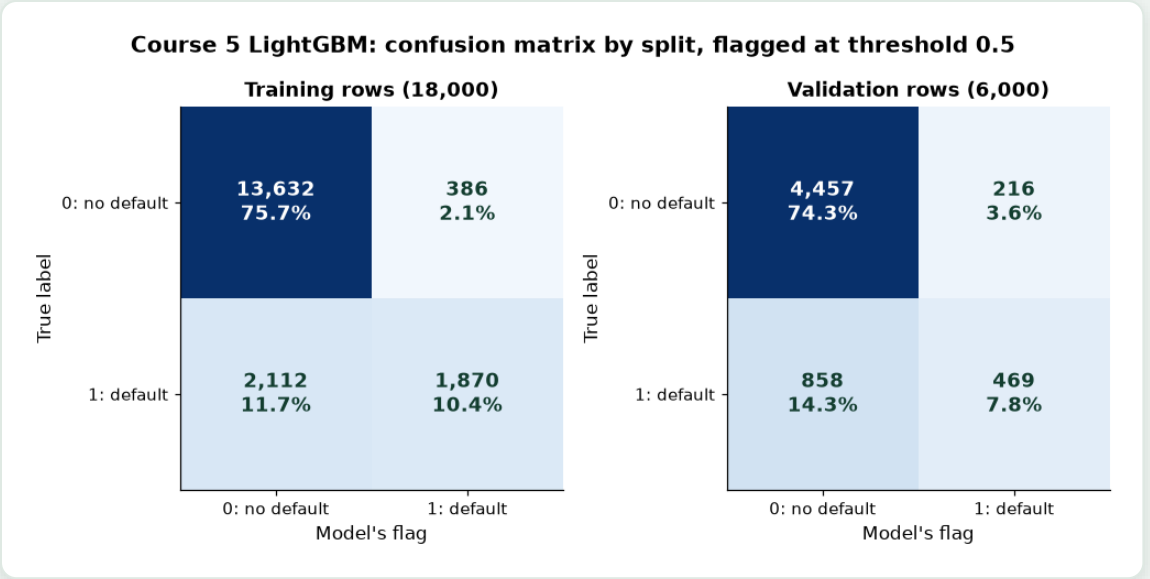
Day 5

A network beside both, on these validation rows, with intervals. The course says what the numbers support and promises nothing.

No network yet today: today is how a network is fitted.

The bar, by split





ROC AUC, training against validation: logistic 0.7296 and 0.7190, LightGBM 0.9220 and 0.7725. The validation numbers are the bar.

One rule for the series: test rows stay closed



fit, and from day 2 stop or choose

report only

closed

Why closed

Course 4 opened them once, on day 10. Scored again after their numbers were seen, they would become rows the course chooses against.

From day 2: fit and stop

14,400 rows fit; a stratified fifth, 3,600 rows, stops or chooses (Course 5's early_stopping_rows). Validation reports once per chosen model.

course4_card_rows never returns a closed row. Course 6 chooses nothing on the validation rows.

Devices: TensorFlow on the CPU

Devices TensorFlow can use here

```
CPU: /physical_device:CPU:0

GPUs: 0
```

TensorFlow's own notice on Windows

"TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11."

Colab: Runtime, Change Runtime Type, Hardware Accelerator

Colab's FAQ: "free of charge access to computing resources, including GPUs and TPUs"; "not guaranteed and not unlimited". Not run or timed for this course.

Every notebook runs on the CPU within its time budget. A GPU is never required.

Sources, read 6 October 2026: tensorflow/python/framework/config.py (TensorFlow 2.21.0); research.google.com/colaboratory/faq.html

How tensors work

1. A block of numbers

any number of axes

2. A shape

the length of each axis

3. A dtype

float32 or float64

4. Reshape

same numbers, new layout; -1 is whatever is left

You control

The shape, the dtype, and the layout.

What comes out

An array like numpy's that TensorFlow can take gradients of.

Tensors: a shape and a dtype

Tensor	Shape	dtype
tf.constant(0.5)	()	float32
PAY_0, training rows	(18000,)	float64
five card rows	(5, 19)	int64

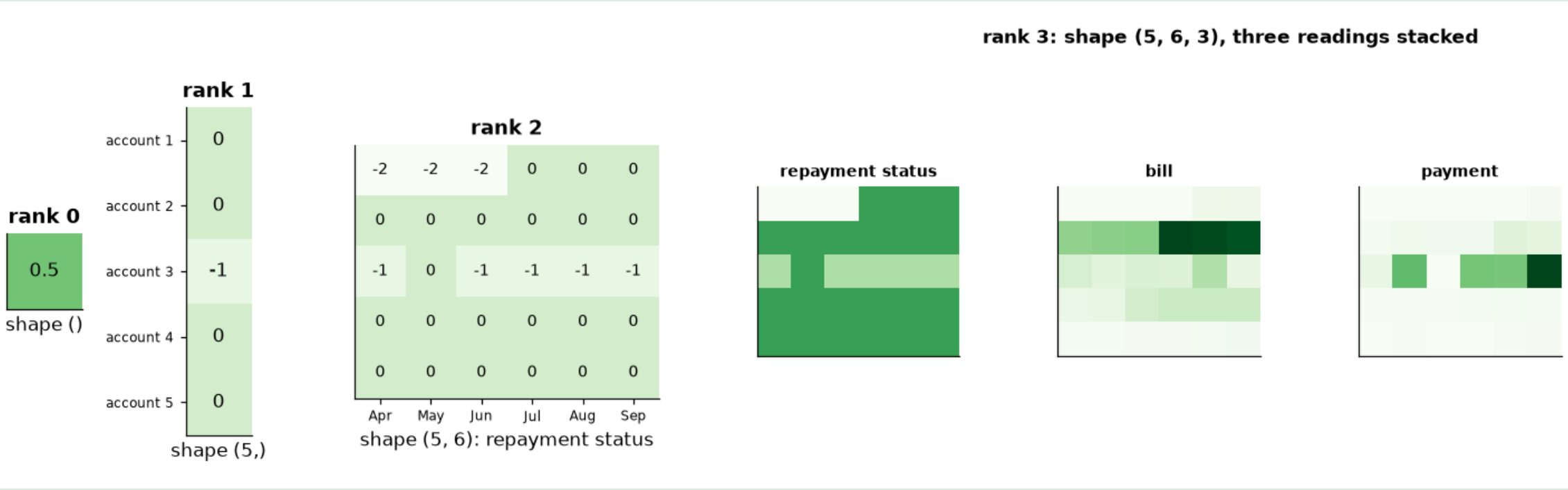
Two precisions

float32: about 7 digits, Keras's default; compared within 1e-6. float64: about 16 digits; compared within 1e-12.

```
one_number * pay_0                    # InvalidArgumentError: float32 times float64
tf.cast(one_number, tf.float64) * pay_0 - 1.0   # runs, element by element
```

Rows first, then columns: a batch of rows is a range of cells.

Tensors by rank, from one number to a stack



Each rank adds one way to find a number. Week 3's sequence networks take stacks of this kind: windows, days, tenors.

How a neuron works

1. Weight each input

multiply by a weight

2. Add an intercept

z = b + w x, a straight line

3. Activation

the sigmoid squeezes z into 0 to 1

4. Score the output

mean log loss over the rows

You control

The weights and the intercept; fitting chooses them.

What comes out

One number per account: the model probability of default in this file.

One neuron by hand

```
z = b + w * x
p = 1 / (1 + exp(-z))
```

One input, one weight, an intercept, the sigmoid: Course 4's logistic regression on one column.

Start: w = 0.5, b = -1.0, on PAY_0

Mean output 0.2795 against a default rate of 0.2212: too high.

0.4876

mean log loss at the start

0.471993

scikit-learn's minimum on PAY_0

Log loss: -(y ln p + (1 - y) ln(1 - p)), averaged. Keras names it binary_crossentropy.

How a gradient works

1. Pick one weight

hold the others fixed

2. Read the slope

of the mean log loss

3. Read the sign

negative: raise it; positive: lower it

4. Let the tape do it

record the arithmetic, replay it backward

You control

Nothing: the loss, the weights, and the rows fix it.

What comes out

One number per weight: the sign and the size are what the next step uses.

The gradient by hand, and by TensorFlow

```
gradient for w = average of (p - y) * x
gradient for b = average of (p - y)
```

-0.033916

weight: negative, so raise w

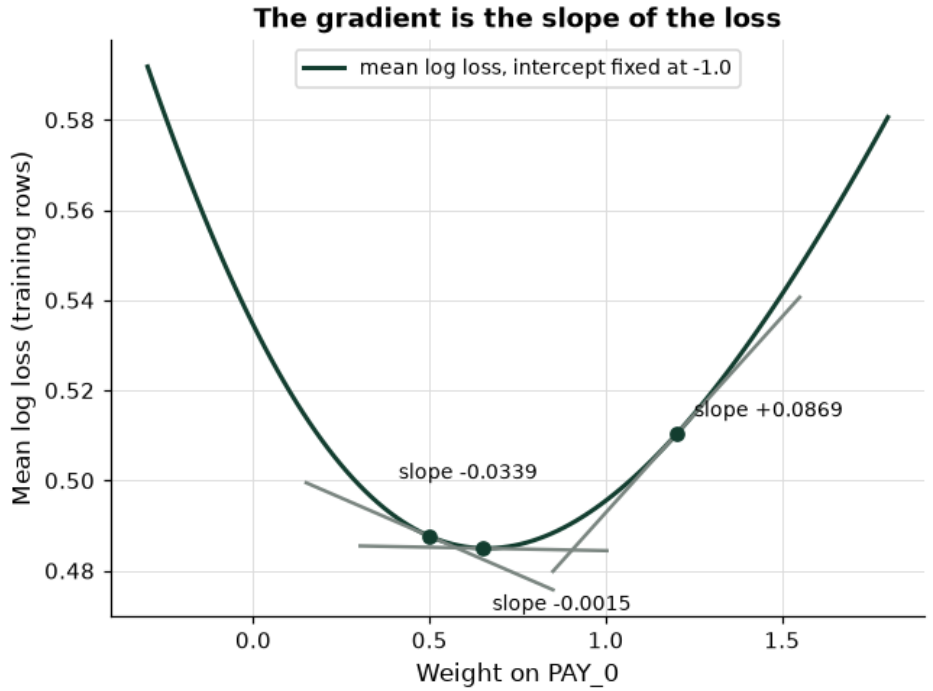
0.058284

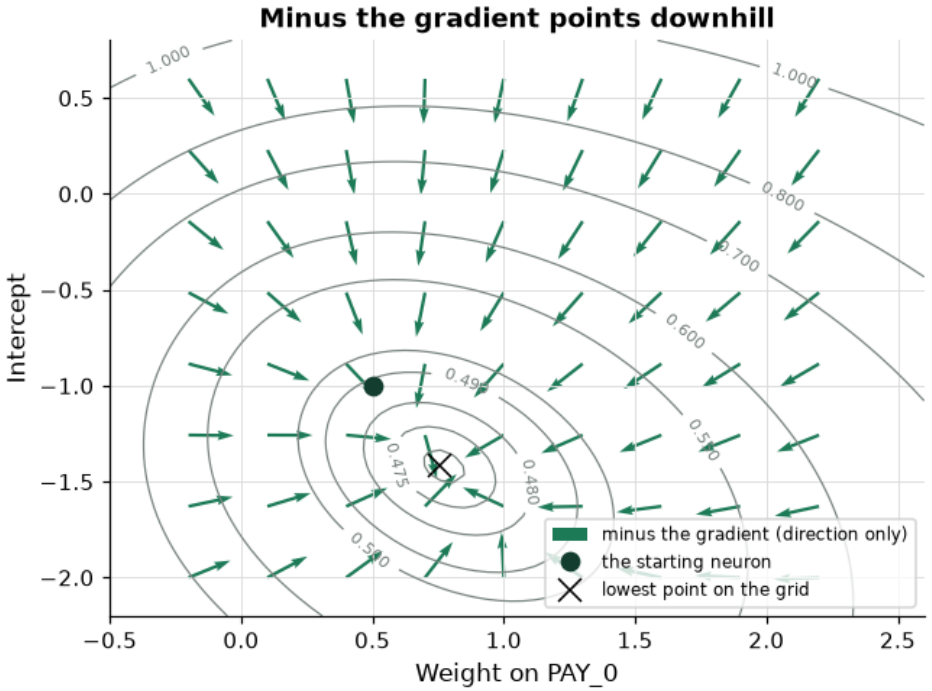
intercept: positive, so lower b

tf.GradientTape finds the same two numbers from the loss alone: within 1e-12 in float64, 1e-6 in float32.

For one neuron the formula is two lines. For thousands of weights, the tape works it out.

The gradient as a slope, and as downhill





Slope at the start -0.0339: raise the weight. Near the bottom -0.0015: flat. Past it +0.0869: lower the weight.

How gradient descent works

1. Gradient

at the current weights

2. Step

w = w - learning rate x gradient

3. Again

the gradient at the new weights

4. Stop

after a set number of steps

You control

The learning rate, the starting weights, and the number of steps.

What comes out

A path of weights and a loss that falls; full batch uses every row.

Gradient descent by hand

```
w = w - learning_rate * gradient        # every weight, then again
```

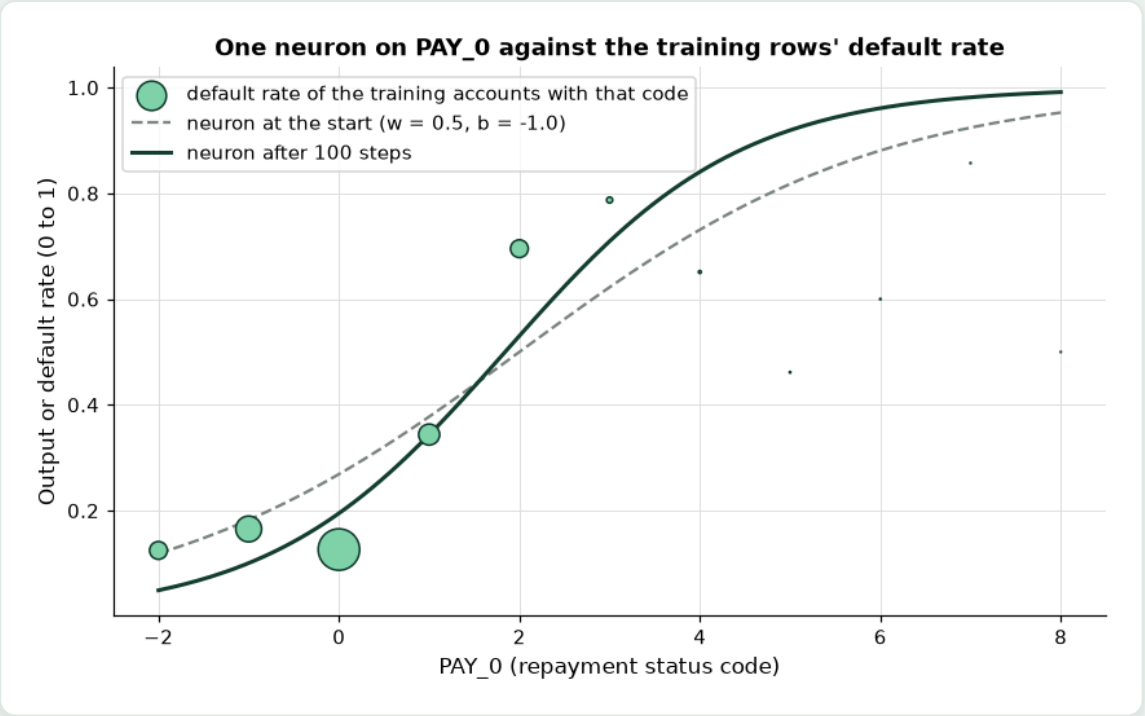
Step	Weight	Mean log loss
0	0.500000	0.487621
1	0.533916	0.483442
10	0.691431	0.472997
50	0.769413	0.471994
100	0.770210	0.471993

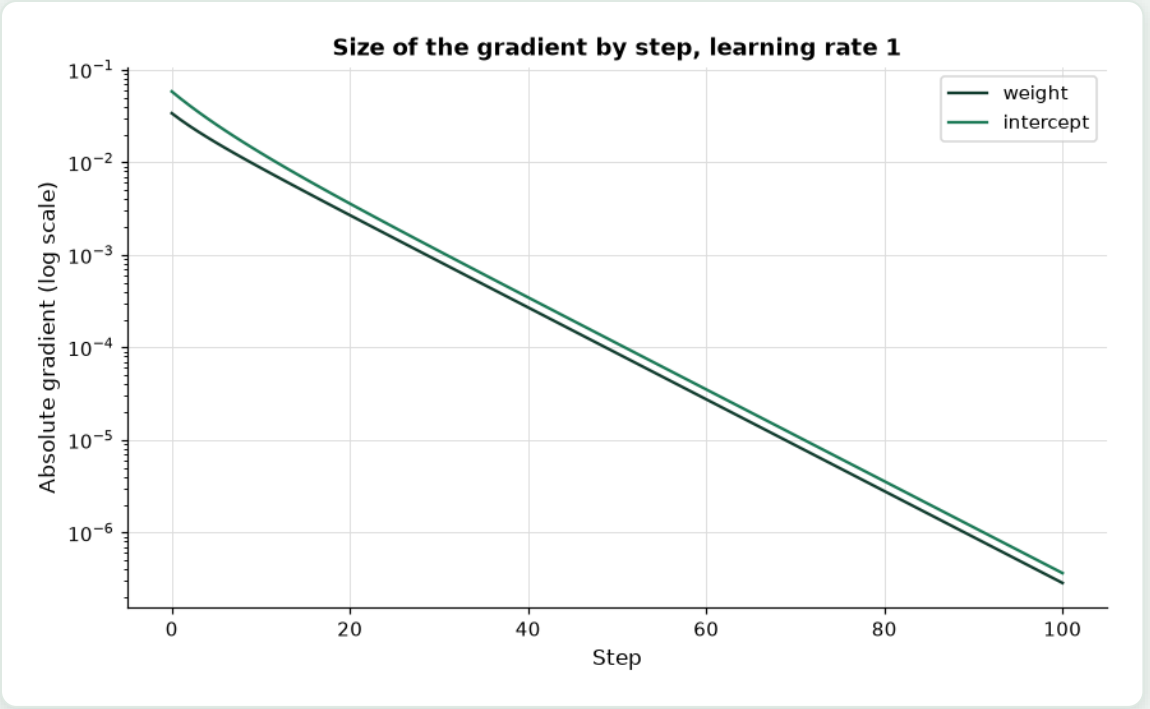
100 steps at learning rate 1, against scikit-learn

By hand: weight 0.770210, intercept -1.418545. Scikit-learn, unpenalised: 0.770212 and -1.418548. Equal within 1e-5; loss 0.471993 both.

One neuron fitted by gradient descent is a logistic regression: day 2.

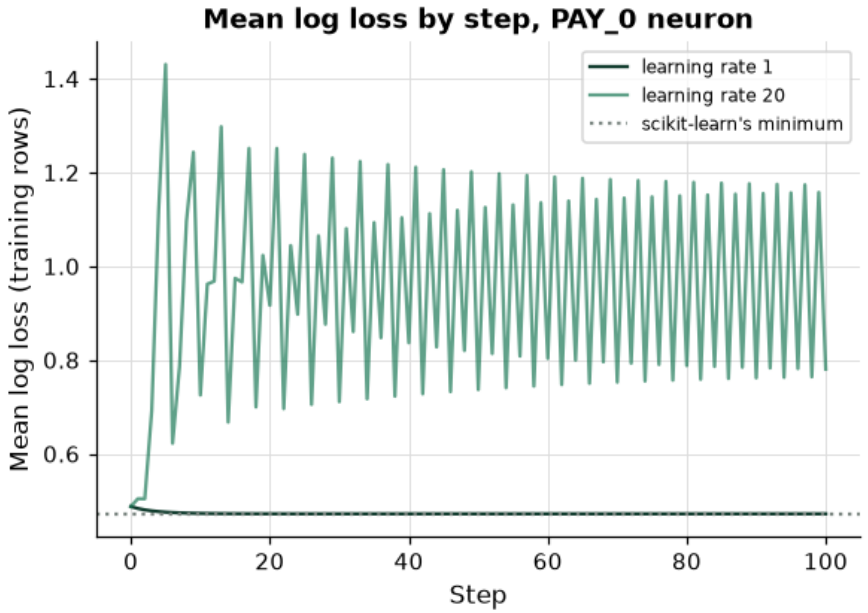
What a hundred steps did

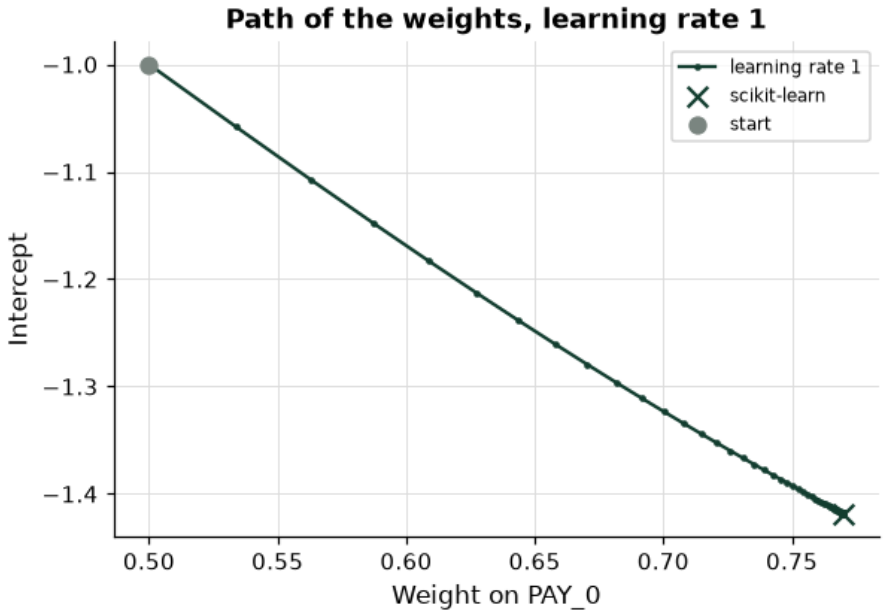




One weight can only draw a rising curve. The gradient shrinks every step, so the steps shrink with it.

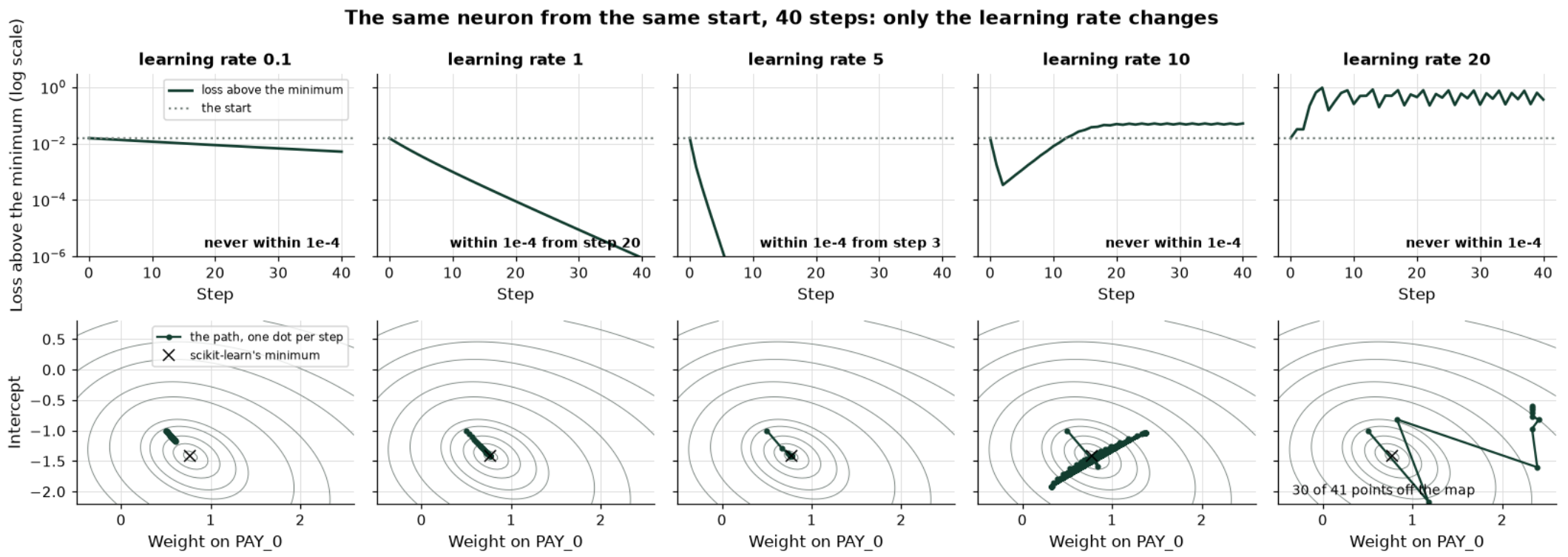
A learning rate that is too large





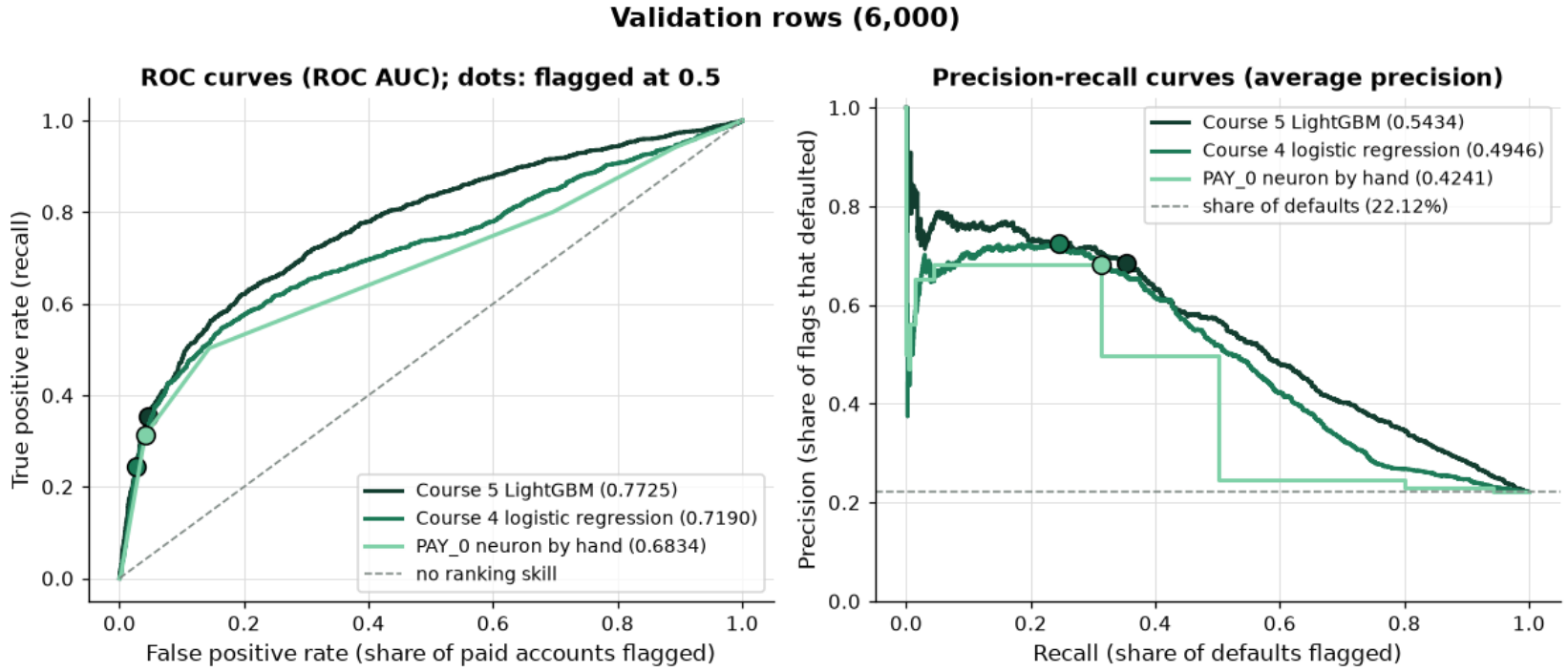
Learning rate 20: 0.4876, then 0.5042 after one step; as high as 1.4305; 0.7805 after 100. Too large never arrives.

Five learning rates, side by side



Rate 0.1 is still 0.0052 above the minimum at step 40. Rate 5 is the fastest clean one; 10 is past the edge; 20 never settles.

Three models on the validation rows



The neuron gives 10 different outputs, one per PAY_0 code: its curves are straight pieces. Dots: flagged at 0.5.

How Keras fits a neuron

1. Build

Dense(1, sigmoid): a weight, an intercept

2. Compile

choose the loss and the optimizer

3. Fit

gradient by tape, then the steps

4. History

the loss after each epoch

You control

Layers, loss, optimizer and learning rate, epochs, batch size.

What comes out

A fitted model and its History.

What Keras automates

```
keras.utils.set_random_seed(SEED)
neuron = keras.Sequential([keras.Input(shape=(1,)), keras.layers.Dense(1, activation="sigmoid")])
neuron.compile(optimizer=keras.optimizers.SGD(learning_rate=1.0), loss="binary_crossentropy")
neuron.fit(X, y, epochs=1, batch_size=len(X), shuffle=False, verbose=0)   # one step
```

After one step	Weight	Intercept
Keras's fit	0.533916	-1.058284
gradient_step	0.533916	-1.058284

What Keras adds, days 2 to 5

A gradient for every weight, mini-batches, epochs, better optimizers, stopping rows.

Equal within 1e-6: the step you wrote by hand is the step inside fit.

The name for what Keras does to every weight

Rumelhart, Hinton, and Williams, October 1986

"Learning representations by back-propagating errors"

Nature, volume 323, issue 6088, pages 533 to 536

Back-propagation: the gradient for every weight, from the output back to the inputs.

Source, read 6 October 2026: the Crossref record of the paper (DOI 10.1038/323533a0). Bibliographic facts only; the paper itself was not read for this course.

Excel side by side

Excel side by side (tested in Excel)

`=1/(1+EXP(-($H$1+$H$2*A2)))`	the neuron, in C2, filled down
`=-(B2*LN(C2)+(1-B2)*LN(1-C2))`	filled down; =AVERAGE(D2:D18001) = 0.487621
`=SUMPRODUCT(C2:C18001-B2:B18001)/ROWS(A2:A18001)`	intercept gradient 0.058284
`=SUMPRODUCT(C2:C18001-B2:B18001,A2:A18001)/ROWS(A2:A18001)`	weight gradient -0.033916
`=$H$1-$H$5*$H$6 and =$H$2-$H$5*$H$7`	one step
`=B2-$I$1*E2 and =C2-$I$1*F2`	100 rows: weight 0.770210

No automatic differentiation and no network training in Excel. Every number above equals Python's within 1e-12.

Save the day: the experiment log

1. Init

The work folder, local settings, the guard

2. .gitignore

adds *.keras: saved networks

3. Code

mlkit, ensemblekit, netkit and its tests

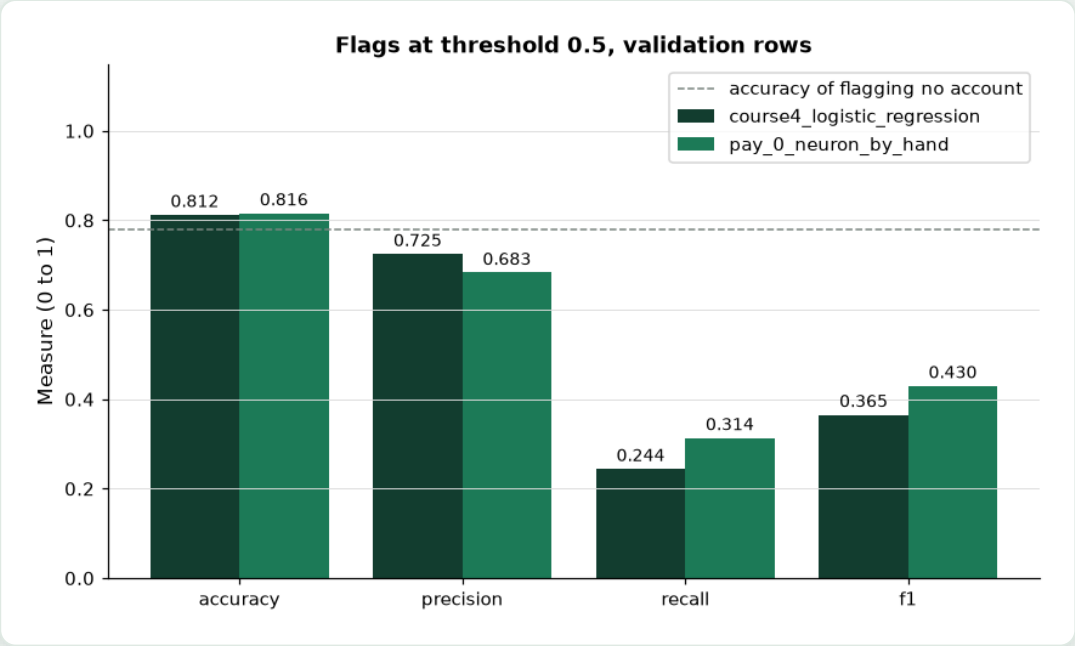
4. Results

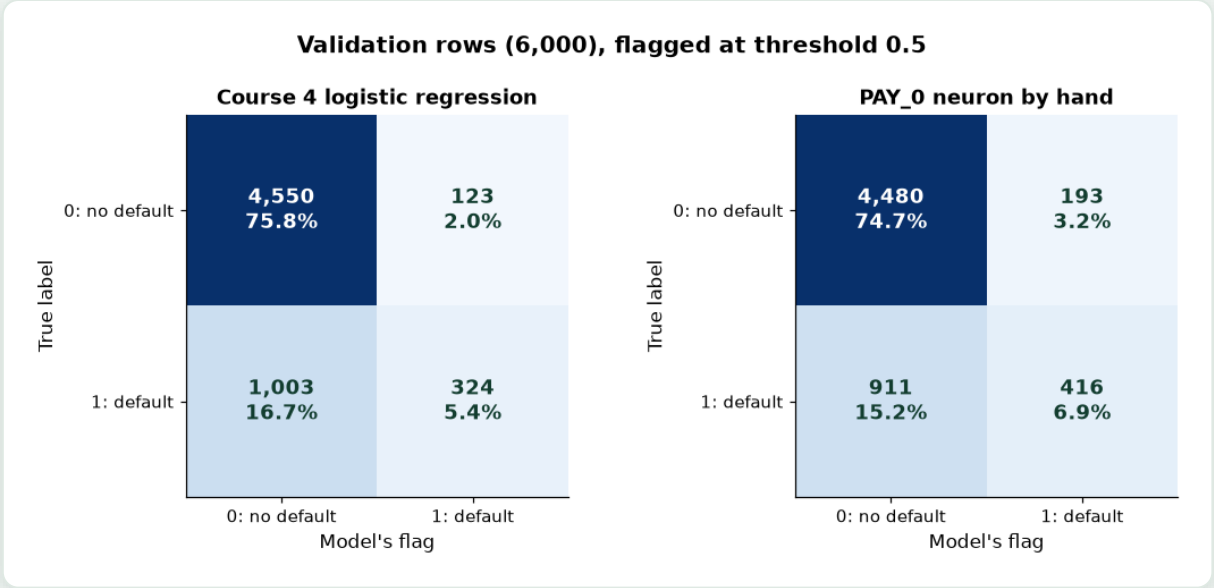
results.csv, 4 decimals

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
course4_logistic_regression,validation,0.7190,0.4946,0.5000,0.8123,0.7248,0.2442,0.3653
pay_0_neuron_by_hand,validation,0.6834,0.4241,0.5000,0.8160,0.6831,0.3135,0.4298
```

Threshold 0.5 today; from day 5, out-of-fold. Accuracy beside the all-zero baseline, 0.7788, never to choose.

Accuracy at 0.5 against ranking





Accuracy at 0.5: neuron 0.8160, logistic 0.8123. ROC AUC: 0.6834 against 0.7190. One threshold is one point on the curve.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the gradient is summed, not averaged: weight gradient -610.4847, loss 2.8860 after one step.

Code that runs is not the same as code that is right.

What you can do now

- Say why TensorFlow runs on the CPU on Windows
- Make, stack, and reshape tensors; read their shape and dtype
- Compute a neuron's output and mean log loss by hand
- Read a gradient as a slope; check it against tf.GradientTape
- Fit one neuron by gradient descent; pick a learning rate by eye

Week 1, Day 2 One neuron is a logistic regression, in Keras